

Plugin za konverziju MATPOWER modela i
statičku analizu toka snage u pravougaonim
koordinatama

Projekat uradio: Emir Krdžalić

July 6, 2026

1 Opis projekta

Analiza toka snage (power flow) je jedan od osnovnih proračuna u elektroenergetici, kojim se za zadano stacionarno stanje mreže (potrošnja, proizvodnja i topologija) određuju naponi na svim sabirnicama. Standardni alat za ovu vrstu proračuna u akademskom i istraživačkom okruženju je MATPOWER, paket za MATLAB koji opisuje mrežu kroz tzv. case fajl (.m). Cilj ovog projekta je da se omogući korištenje ovakvih modela direktno u simulacionom okruženju dTwin, bez potrebe za instaliranim MATLAB-om.

U tu svrhu razvijen je dTwin plugin (interni naziv *mpconv4*, stavka u meniju “RctPlugin”) koji:

- učitava i parsira MATPOWER .m fajl (blokove `mpc.bus`, `mpc.gen` i `mpc.branch`),
- na osnovu podataka o vodovima i transformatorima gradi kompleksnu admitansnu matricu mreže (Ybus),
- generiše sistem nelinearnih algebarskih jednačina (NLEs) u pravougaonim koordinatama napona, i
- zapisuje sve u .dmodl fajl koji dTwin učitava i rješava internim Newton-Raphson rješavačem.

Pravougaone koordinate napona su izabrane umjesto polarnih (magnituda/ugao) jer, kao što će biti pokazano u nastavku, vode do jednačina koje su algebarski jednostavnije (polinomske, bez trigonometrijskih funkcija), što je pogodnije za formulaciju u dTwin NLE modelu.

Cjelokupna arhitektura sistema (parser → gradnja Ybus → generisanje .dmodl → rješavanje u dTwin-u) prikazana je na slici 1.

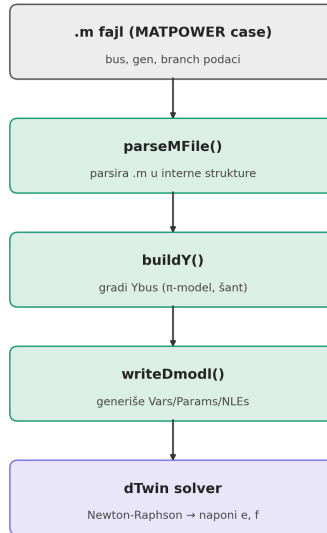


Figure 1: Arhitektura plugina: tok podataka od .m fajla do rješenja u dTwin-u

2 Cilj projekta

Glavni cilj projekta je razvoj plugina i grafičkog interfejsa (GUI) za dTwin koji omogućava:

- konverziju proizvoljnog MATPOWER case fajla u dTwin model (.dmodl) bez posrednog koraka kroz MATLAB,
- rješavanje statičkog toka snage u pravougaonim koordinatama, primjenom Newton-Raphsonove metode koju interno koristi dTwin,
- jednostavno grafičko korisničko okruženje kroz koje se biraju ulazni/izlazni fajlovi i podešavaju parametri rješavača (broj iteracija, tolerancija konvergencije), i
- provjeru ispravnosti implementacije poređenjem dobijenih napona sa referentnim MATPOWER rezultatima na standardnim IEEE test sistemima.

Fokus rada je, dakle, na dvije cjeline: matematičkom modelu koji stoji iza konverzije (formiranje Ybus matrice i NLE jednačina) i softverskoj implementaciji plugina, uključujući GUI kroz koji korisnik upravlja cijelim procesom.

3 Matematički model toka snage u pravougaonim koordinatama

3.1 Reprezentacija napona i osnovna jednačina snage

Za svaku sabirnicu i napon se u pravougaonim (Kartezijevim) koordinatama zapisuje kao

$$V_i = e_i + jf_i, \quad e_i = \text{Re}(V_i), \quad f_i = \text{Im}(V_i)$$

Injektovana kompleksna snaga na sabirnici i data je poznatim izrazom

$$S_i = P_i + jQ_i = V_i I_i^*, \quad I_i = \sum_j Y_{ij} V_j$$

gdje je $Y_{ij} = G_{ij} + jB_{ij}$ element admitansne matrice mreže (Ybus), čija je gradnja objašnjena u narednoj podsekciji. Cilj je izraziti P_i i Q_i direktno preko e i f komponenti, čime se dobijaju jednačine koje su pogodne za Newton-Raphson rješavač.

3.2 Formiranje matrice admitansi mreže (Ybus)

Ybus matrica se formira jednom, u pred-obradi (funkcija `buildY`), iz podataka o vodovima i transformatorima (blok `mpc.branch`), koristeći opći π -model grane sa transformatorskim odnosom. Neka je $\mathcal{E}$ skup svih grana (vodova i transformatora) mreže. Za granu koja povezuje sabirnice p i q (u kodu ovo su indeksi `fi` i `ti`), sa serijskom impedansom $r_{pq} + jx_{pq}$, šant susceptansom b_{pq} i kompleksnim odnosom transformacije $t_{pq} = a_{pq} e^{j\varphi_{pq}}$ (magnituda a_{pq} i fazni pomak φ_{pq} su MATPOWER polja `tap` i `shift`), definišu se pomoćne veličine

$$y_{pq} = \frac{1}{r_{pq} + jx_{pq}}, \quad y_{pq}^{sh} = \frac{j b_{pq}}{2}, \quad t_{pq} = a_{pq} e^{j\varphi_{pq}}$$

Grana (p, q) tako dobija svoju primitivnu (2×2) admitansnu matricu:

$$\mathbf{Y}_{bus} = \sum_{k=1}^m Y_k^{(pq)}$$

Ukupna Ybus matrica mreže dobija se sumiranjem (superpozicijom) primitivnih matrica svih grana:

$$\mathbf{Y}_{pp} = \sum_{(p,q) \in \mathcal{E}} Y^{(pq)}$$

Gledano po elementima, dijagonalni element sabirnice p je zbir doprinosa svih grana koje se u njoj sijeku, uz dodatak vlastite šant admitanse sabirnice (blok `mpc.bus`, kolone `Gs`, `Bs`), svedene na baznu snagu (`baseMVA`):

$$Y_{pp} = \sum_{q:(p,q) \in \mathcal{E}} Y_{pq}^{(pq)} + \frac{G_{s,p}}{baseMVA} + j \frac{B_{s,p}}{baseMVA}$$

Ovim postupkom, ponavljanim za sve grane i sve sabirnice, dobija se kompletna, rijetka (sparse) admitansna matrica $Y_{ij} = G_{ij} + jB_{ij}$, koja se u kodu čuva u strukturi `sparse::ICmplxMatrix` i dalje se ne mijenja tokom iterativnog rješavanja.

3.3 Izvođenje jednačina snage u pravougaonim koordinatama

Polazeći od $I_i = \sum_j Y_{ij} V_j$ i uvrštavanjem $Y_{ij} = G_{ij} + jB_{ij}$, $V_j = e_j + jf_j$, konjugovana struja glasi

$$I_i^* = \sum_j [(G_{ij}e_j - B_{ij}f_j) - j(G_{ij}f_j + B_{ij}e_j)]$$

Uvrštavanjem u $S_i = (e_i + jf_i) \cdot I_i^*$ i razdvajanjem realnog i imaginarnog dijela, dobijaju se opće jednačine injektovane aktivne i reaktivne snage, izražene isključivo preko e i f komponenti napona:

$$P_i = \sum_j [G_{ij}(e_i e_j + f_i f_j) - B_{ij}(e_i f_j - f_i e_j)]$$

$$Q_i = \sum_j [G_{ij}(f_i e_j - e_i f_j) - B_{ij}(e_i e_j + f_i f_j)]$$

Ove jednačine su isključivo polinomske (kvadratne) po e i f , bez sinusa i kosinusa koji bi se javili u polarnoj formulaciji — to je i osnovni razlog zašto je plugin implementiran u pravougaonim, a ne u polarnim koordinatama.

3.4 Nelinearne jednačine (NLEs) po tipu sabirnice

Funkcija `writeDmodl` iz gore izvedenih opštih izraza generiše konkretne jednačine za svaku sabirnicu, u zavisnosti od njenog tipa (kolona `type` u `mpc.bus`). Prije toga, iz podataka o generatorima i potrošnji (`mpc.gen`, `mpc.bus`), svedenih na baznu snagu, računaju se poznate injekcije:

$$P_i^{sp} = \frac{\sum P_g - P_{d,i}}{baseMVA}, \quad Q_i^{sp} = \frac{\sum Q_g - Q_{d,i}}{baseMVA}$$

- **PQ sabirnica** (tip 1, potrošačka) — nepoznate su e_i i f_i , pa se postavljaju dvije jednačine (aktivna i reaktivna snaga moraju biti jednake zadanim vrijednostima):

$$P_i = P_i^{sp}, \quad Q_i = Q_i^{sp}$$

- **PV sabirnica** (tip 2, generatorska sa regulacijom napona) — poznata je proizvedena aktivna snaga i zadana magnituda napona V_i^{sp} , pa se postavlja jednačina aktivne snage i uslov fiksirane magnitude napona (reaktivna snaga Q_i se ne fiksira, već se dobija u post-obradi):

$$P_i = P_i^{sp}, \quad e_i^2 + f_i^2 = (V_i^{sp})^2$$

- **Slack (balansna) sabirnica** (tip 3) — magnituda i ugao napona su fiksirani i unaprijed poznati, pa e_{slack} i f_{slack} nisu nepoznate, već se u .dmodl fajlu zapisuju kao **Params** i ne ulaze u sistem NLE jednačina. Ova sabirnica pokriva razliku (gubitke) u ukupnom bilansu snage mreže.

Sistem koji rješava dTwin sadrži, dakle, po dvije jednačine za svaku PQ sabirnicu i po dvije za svaku PV sabirnicu (ukupno $2(n-1)$ jednačina za mrežu sa n sabirnica, od kojih je jedna slack), što je tačno onoliko koliko je nepoznatih e_i, f_i komponenti. Nakon konvergencije, u post-obradi se iz dobijenih napona računaju izvedene veličine: reaktivna snaga generatora na PV sabirnicama, snaga slack sabirnice (koja pokriva gubitke) i polarni oblik napona (magnituda i ugao) radi lakše interpretacije rezultata.

4 Arhitektura i implementacija koda

Plugin je organizovan kao natID/dTwin dodatak koji se kompajlira u dinamičku biblioteku `mpconv4.dll` (CMake target definisan u `MatPowerPlugin4.cmake`, solution `MatPowerPlugin4Sol` definisan u `CMakeLists.txt`). Struktura izvornog koda je sljedeća:

- `src/MatPowerPlugin.h` — definicija strukture `Options` (podešavanja rješavača) i `PLUGIN_API` makroa;
- `src/MatPowerPlugin.cpp` — parser `MATPOWER` fajla, gradnja `Ybus` matrice, generisanje NLE jednačina i implementacija plugin klase;
- `src/ViewConv.h` — GUI Converter taba;
- `src/ViewOptions.h` — GUI Options taba;
- `src/TabView.h` — kontejner koji objedinjuje oba taba;
- `src/WindowPlugin.h` — glavni prozor plugina.

Plugin prema dTwin-u nastupa kroz standardni interfejs `sc::IPlugin`, pri čemu metoda `getMenuName()` vraća naziv koji se prikazuje u meniju dTwin-a (“Ret-Plugin”). Za rad sa fajlovima i memorijom koriste se `natID` biblioteke `fo::*` (čitanje/pisanje fajlova), `arch::MemoryOut` (in-memory bafer za .dmodl sadržaj), `cnt::PushBackVector` (tokenizacija) i `mu::ScopedCLocale` (ispravno parsiranje decimalnih brojeva bez obzira na regionalne postavke sistema).

4.1 Parsiranje MATPOWER fajla — `parseMFile`

MATPOWER .m fajl se čita liniju po liniju (`fo::InFile`, `fo::getLine`) i traže se tri karakteristična bloka: `mpc.bus`, `mpc.gen` i `mpc.branch`. Za svaki blok se iz svake linije tokenizacijom izvlače numerički podaci u interne strukture:

- **Bus:** identifikator, tip sabirnice (1 = PQ, 2 = PV, 3 = slack), aktivna/reaktivna potrošnja P_d , Q_d , šant provodnost/susceptansa G_s , B_s , te početna magnituda i ugao napona V_m , V_a ;
- **Gen:** sabirnica na kojoj se generator nalazi, proizvedena aktivna/reaktivna snaga P_g , Q_g i zadani napon V_g ;
- **Branch:** sabirnice from/to, parametri r, x, b , odnos transformacije tap , fazni pomak $shift$ i status grane (u/van pogona).

Bitno je napomenuti da MATLAB nije potreban ni u jednom trenutku — plugin direktno parsira tekstualni sadržaj .m fajla, što ga čini potpuno samostalnim alatom.

4.2 Gradnja Ybus matrice — `buildY`

Na osnovu podataka o granama i sabirnicama, funkcija `buildY` primjenjuje formule izvedene u sekciji 1 (π -model, šant admitanse) i popunjava rijetku kompleksnu matricu tipa `sparse::ICmplxMatrix`, uz RAII upravljanje resursima kroz `sparse::CmplxMatrixReleaser`. Matrica se računa samo jednom, prije generisanja jednačina, jer se njeni elementi ne mijenjaju tokom iterativnog Newton-Raphson postupka koji se izvršava kasnije, unutar `dTwin-a`.

4.3 Generisanje .dmodl fajla — `writeDmodl`

Poslednji korak konverzije je zapisivanje kompletnog modela u `dTwin-ov` tekstualni format (.dmodl), kroz in-memory bafer `arch::MemoryOut` koji se na kraju snima na disk pomoću `fo::createTextFile`. Fajl je organizovan u sljedeće blokove:

- **Header** — globalna podešavanja rješavača (`maxIter`, `report`, `maxReps`, `outToTxt`), preuzeta iz Options taba GUI-a;
- **Vars** — nepoznate e_i, f_i za sve sabirnice osim slack, sa početnom pretpostavkom (obično $e_i = 1, f_i = 0$) potrebnom iterativnom rješavaču;
- **Params** — fiksni slack napon, svi elementi Ybus matrice (G_{i-j} , B_{i-j}) i sve poznate injekcije snaga (P_i , Q_i , $Vsp2_i$);
- **NLEs** — konkretne jednačine iz sekcije 3.4, po jedna, odnosno dvije, za svaku sabirnicu ovisno o njenom tipu.

Ovakog generisanog .dmodl fajla `dTwin` učitava i rješava svojim internim Newton-Raphson rješavačem, pri čemu su `Max iters` i `eps` (podešeni kroz GUI) direktna podešavanja tog rješavača.

5 Grafički interfejs (GUI) i uputstvo za korištenje

Nakon uspješnog build-a (Release x64) i kopiranja `mpconv4.dll` u folder dTwin plugin-a, u glavnom meniju dTwin-a se pojavljuje nova stavka, “RctPlugin” (slika 2), čijim se pokretanjem otvara glavni prozor plugina (WindowPlugin, dimenzija 800x300), sa dva taba: *Converter* i *Options*.

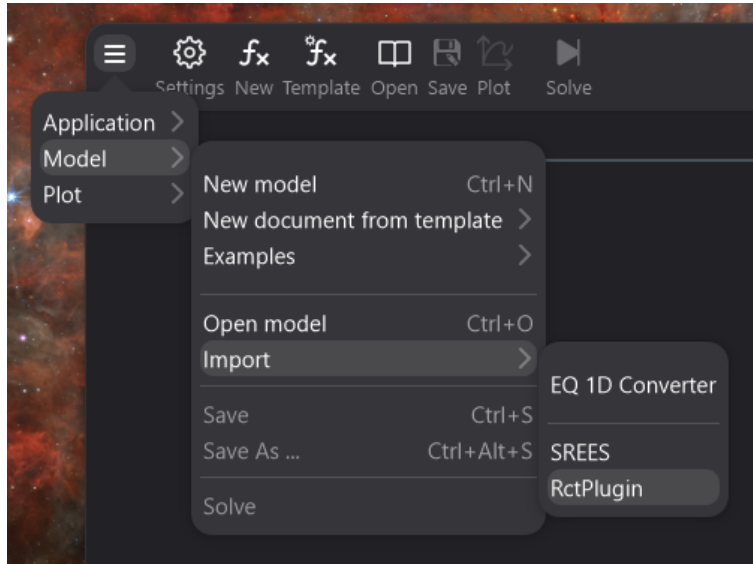


Figure 2: Stavka “RctPlugin” u glavnom meniju dTwin-a

5.1 Converter tab

Converter tab je glavni radni prostor plugina i sadrži:

- polje **Import model (.m)** sa dugmetom “...” za odabir MATPOWER case fajla — odabirom fajla plugin automatski prelazi u **Convert** mod;
- polje **Load model / Save as** sa dugmetom “...”, čija se labela dinamički mijenja: u Convert modu postaje “Save as:” (upisuje se ime izlaznog .dmodl fajla), a kada je ovo polje prazno, ostaje “Load model:” (za odabir postojećeg .dmodl fajla);
- statusnu liniju sa porukom o uspjehu ili grešci posljednje operacije;
- dugmad **Info** (provjera postojanja odabranog fajla), **Load** (učitavanje postojećeg .dmodl direktno u dTwin, aktivno samo u Load modu), **Convert** (parsiranje .m, gradnja Ybus, generisanje i automatsko učitavanje .dmodl, aktivno samo u Convert modu) i **Cancel** (brisanje svih polja i povratak u početno stanje).

U nastavku su prikazana tri karakteristična stanja Converter taba: početno stanje, stanje nakon odabira ulaznog .m fajla (Convert mod) i stanje kada je odabran postojeći model za učitavanje (Load mod).

Na slici 3 prikazano je početno stanje Converter taba, prije bilo kakvog odabira fajla — oba polja su prazna, a dugmad **Load** i **Convert** su neaktivna.

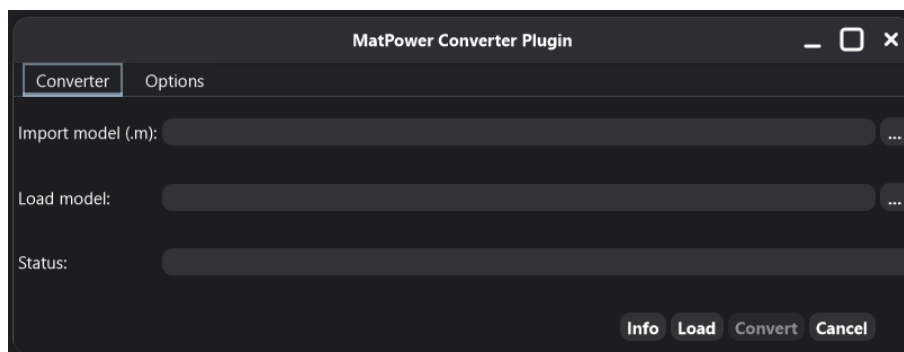


Figure 3: Početno stanje Converter taba (prazna polja)

Na slici 4 prikazan je Converter tab nakon što je u polje **Import model (.m)** odabran ulazni MATPOWER case fajl. Plugin je automatski prešao u **Convert** mod, pa se labela drugog polja promijenila u “Save as:”, a dugme **Convert** je postalo aktivno.

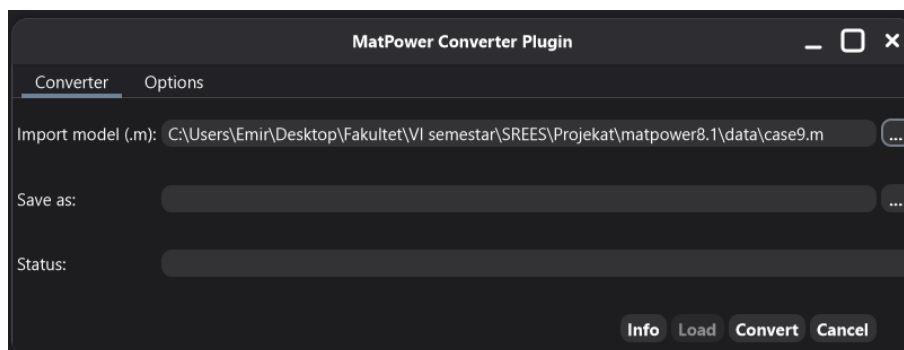


Figure 4: Converter tab u Convert modu — odabran ulazni .m fajl, polje za izlaz postaje “Save as:”

Na slici 5 prikazan je Converter tab u **Load** modu: polje “Import model (.m):” je prazno, dok je u drugo polje (labela “Load model:”) odabran postojeći .dmodl fajl, čime se aktivira dugme **Load** za direktno učitavanje modela u dTwin, bez ponovne konverzije.

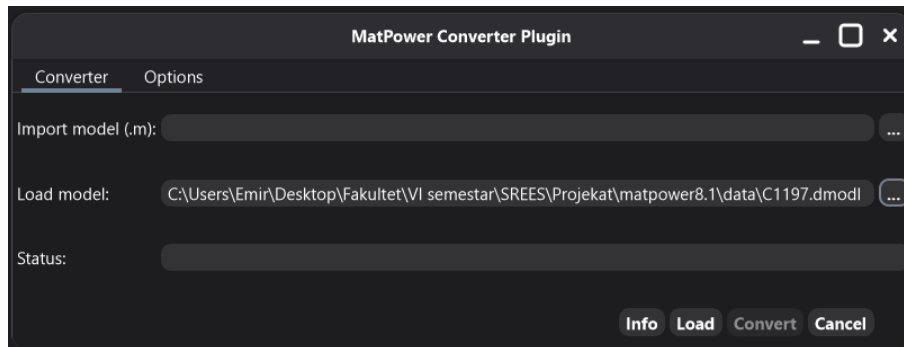


Figure 5: Converter tab u Load modu — odabran postojeći .dmodl fajl za učitavanje

5.2 Options tab

Options tab (slika 6) sadrži podešavanja koja se upisuju u header generisanog .dmodl fajla:

- **Model name** — naziv koji se upisuje u header .dmodl fajla;
- **Max iters** — maksimalan broj Newton-Raphson iteracija (podrazumijevano 100);
- **eps** — tolerancija konvergencije rješavača (podrazumijevano 10^{-6}).

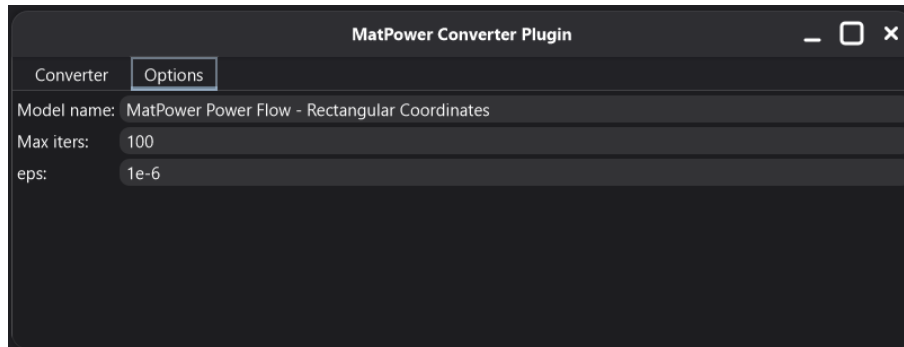


Figure 6: Izgled Options taba

5.3 Tipičan tok rada

A) Nova konverzija (.m → .dmodl): odabere se ulazni .m fajl (npr. `case14.m`), plugin automatski prelazi u Convert mod, upiše se ime izlaznog fajla (ili se odabere lokacija preko "..."), po potrebi se podese Options, te se klikne **Convert** — dTwin automatski učitava i rješava dobijeni model.

B) Učitavanje postojećeg modela: u polje “Load model:” upiše se (ili odabere) putanja do postojećeg .dmodl fajla, te se klikne **Load** — model se učitava bez ponovne konverzije.

C) Resetovanje: klikom na **Cancel** se čiste sva polja i plugin se vraća u početno stanje.

6 Testiranje i rezultati

Implementacija je testirana na dva standardna IEEE test sistema: **case9.m** (9 sabirnica) i **case14.m** (14 sabirnica). U oba slučaja rješavač je konvergirao za samo 2 iteracije, a dobijeni naponi (magnituda i ugao) se poklapaju sa referentnim rezultatima MATPOWER-a, čime je potvrđena ispravnost implementiranog matematičkog modela (gradnje Ybus matrice i NLE jednačina) i samog parsera.

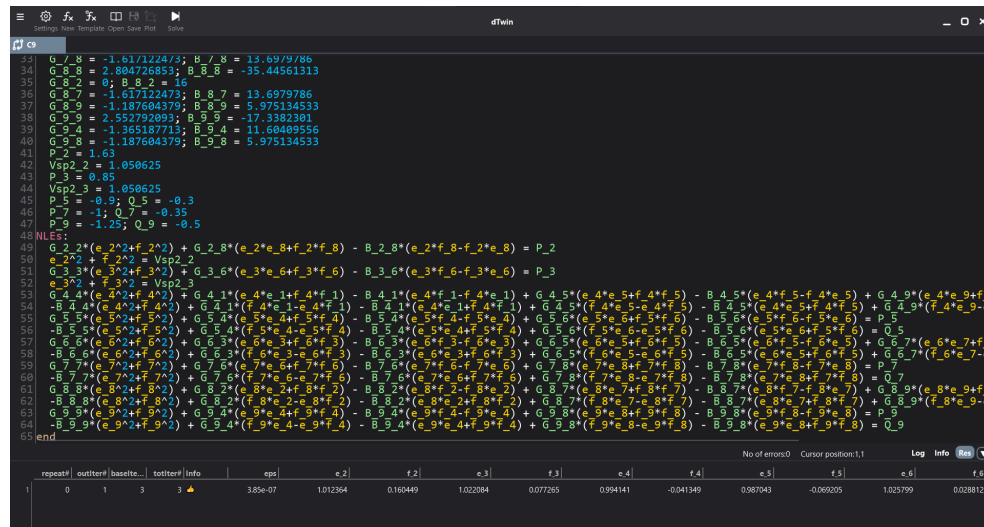


Figure 7: Rezultati konverzije i rješavanja za **case9.m** (IEEE 9-bus) u dTwin-u

Tabela 1 sumira dobijene vrijednosti napona u pravougaonim koordinatama (e, f), direktno kako je ispisano iz dTwin-a, za svih devet sabirnica sistema **case9.m**.

Table 1: Rezultati napona po sabirnicama za case9.m

Sabirnica	e [p.u.]	f [p.u.]
1 (slack)	1	0
2	1.01236	0.160449
3	1.02208	0.0772647
4	0.994141	-0.0413491
5	0.987043	-0.0692055
6	1.0258	0.0288117
7	1.00921	0.00648792
8	1.0173	0.0606009
9	0.96908	-0.0732496

```

c9
P_10 = -0.09; Q_10 = -0.058
P_11 = -0.035; Q_11 = -0.018
P_12 = -0.051; Q_12 = -0.016
P_13 = -0.135; Q_13 = -0.058
P_14 = -0.149; Q_14 = -0.05
NLS:
G_2_2*(e_2^2+f_2^2) + G_2_1*(e_2*e_1+f_2*f_1) - B_2_1*(e_2*f_1-f_2*e_1) + G_2_3*(e_2*e_3+f_2*f_3) - B_2_3*(e_2*f_3-f_2*e_3) + G_2_4*(e_2*e_4+f_2*f_4) - B_2_4*(e_2*f_4-f_2*e_4) = P_2
e_2^2 + f_2^2 = Vsp2_2
G_3_3*(e_3^2+f_3^2) + G_3_2*(e_3*e_2+f_3*f_2) - B_3_2*(e_3*f_2-f_3*e_2) + G_3_4*(e_3*e_4+f_3*f_4) - B_3_4*(e_3*f_4-f_3*e_4) = P_3
e_3^2 + f_3^2 = Vsp2_3
G_4_4*(e_4^2+f_4^2) + G_4_2*(e_4*e_2+f_4*f_2) - B_4_2*(e_4*f_2-f_4*e_2) + G_4_3*(e_4*e_3+f_4*f_3) - B_4_3*(e_4*f_3-f_4*e_3) + G_4_5*(e_4*e_5+f_4*f_5) - B_4_5*(e_4*f_5-f_4*e_5) = P_4
e_4^2 + f_4^2 = Vsp2_4
G_5_5*(e_5^2+f_5^2) + G_5_1*(e_5*e_1+f_5*f_1) - B_5_1*(e_5*f_1-f_5*e_1) + G_5_2*(e_5*e_2+f_5*f_2) - B_5_2*(e_5*f_2-f_5*e_2) + G_5_4*(e_5*e_4+f_5*f_4) - B_5_4*(e_5*f_4-f_5*e_4) = P_5
e_5^2 + f_5^2 = Vsp2_5
G_6_6*(e_6^2+f_6^2) + G_6_5*(e_6*e_5+f_6*f_5) - B_6_5*(e_6*f_5-f_6*e_5) + G_6_11*(e_6*e_11+f_6*f_11) - B_6_11*(e_6*f_11-f_6*e_11) + G_6_12*(e_6*e_12+f_6*f_12) - B_6_12*(e_6*f_12-f_6*e_12) = P_6
e_6^2 + f_6^2 = Vsp2_6
G_7_7*(e_7^2+f_7^2) + G_7_4*(e_7*e_4+f_7*f_4) - B_7_4*(e_7*f_4-f_7*e_4) + G_7_8*(e_7*e_8+f_7*f_8) - B_7_8*(e_7*f_8-f_7*e_8) + G_7_9*(e_7*e_9+f_7*f_9) - B_7_9*(e_7*f_9-f_7*e_9) = P_7
e_7^2 + f_7^2 = Vsp2_7
G_8_8*(e_8^2+f_8^2) + G_8_7*(e_8*e_7+f_8*f_7) - B_8_7*(e_8*f_7-f_8*e_7) = P_8
e_8^2 + f_8^2 = Vsp2_8
G_9_9*(e_9^2+f_9^2) + G_9_4*(e_9*e_4+f_9*f_4) - B_9_4*(e_9*f_4-f_9*e_4) + G_9_7*(e_9*e_7+f_9*f_7) - B_9_7*(e_9*f_7-f_9*e_7) + G_9_10*(e_9*e_10+f_9*f_10) - B_9_10*(e_9*f_10-f_9*e_10) = P_9
e_9^2 + f_9^2 = Vsp2_9
G_10_10*(e_10^2+f_10^2) + G_10_9*(e_10*e_9+f_10*f_9) - B_10_9*(e_10*f_9-f_10*e_9) + G_10_11*(e_10*e_11+f_10*f_11) - B_10_11*(e_10*f_11-f_10*e_11) + G_10_12*(e_10*e_12+f_10*f_12) - B_10_12*(e_10*f_12-f_10*e_12) = P_10
e_10^2 + f_10^2 = Vsp2_10
G_11_11*(e_11^2+f_11^2) + G_11_6*(e_11*e_6+f_11*f_6) - B_11_6*(e_11*f_6-f_11*e_6) + G_11_10*(e_11*e_10+f_11*f_10) - B_11_10*(e_11*f_10-f_11*e_10) + G_11_12*(e_11*e_12+f_11*f_12) - B_11_12*(e_11*f_12-f_11*e_12) = P_11
e_11^2 + f_11^2 = Vsp2_11
G_12_12*(e_12^2+f_12^2) + G_12_6*(e_12*e_6+f_12*f_6) - B_12_6*(e_12*f_6-f_12*e_6) + G_12_13*(e_12*e_13+f_12*f_13) - B_12_13*(e_12*f_13-f_12*e_13) + G_12_14*(e_12*e_14+f_12*f_14) - B_12_14*(e_12*f_14-f_12*e_14) = P_12
e_12^2 + f_12^2 = Vsp2_12
G_13_13*(e_13^2+f_13^2) + G_13_6*(e_13*e_6+f_13*f_6) - B_13_6*(e_13*f_6-f_13*e_6) + G_13_12*(e_13*e_12+f_13*f_12) - B_13_12*(e_13*f_12-f_13*e_12) + G_13_14*(e_13*e_14+f_13*f_14) - B_13_14*(e_13*f_14-f_13*e_14) = P_13
e_13^2 + f_13^2 = Vsp2_13
G_14_14*(e_14^2+f_14^2) + G_14_9*(e_14*e_9+f_14*f_9) - B_14_9*(e_14*f_9-f_14*e_9) + G_14_13*(e_14*e_13+f_14*f_13) - B_14_13*(e_14*f_13-f_14*e_13) + G_14_15*(e_14*e_15+f_14*f_15) - B_14_15*(e_14*f_15-f_14*e_15) = P_14
e_14^2 + f_14^2 = Vsp2_14
end

```

Figure 8: Rezultati konverzije i rješavanja za case14.m (IEEE 14-bus) u dTwin-u

Tabela 2 sumira dobijene vrijednosti napona u pravougaonim koordinatama (e, f), direktno kako je ispisano iz dTwin-a, za svih 14 sabirnica sistema case14.m.

Table 2: Rezultati napona po sabirnicama za `case14.m`

Sabirnica	e [p.u.]	f [p.u.]
1 (slack)	1.06	0
2	1.04105	-0.0907614
3	0.985193	-0.222476
4	1.00123	-0.182187
5	1.00758	-0.155511
6	1.03721	-0.262858
7	1.03279	-0.245277
8	1.0605	-0.251858
9	1.02024	-0.272201
10	1.01471	-0.273738
11	1.02189	-0.269815
12	1.01887	-0.274447
13	1.01385	-0.274625
14	0.995247	-0.286015

7 Zaključak

Razvijeni plugin uspješno povezuje MATPOWER format opisa elektroenergetske mreže sa dTwin simulacionim okruženjem, bez potrebe za MATLAB-om. Izvođenjem jednačina toka snage u pravougaonim koordinatama dobijen je isključivo polinomski sistem NLE jednačina, pogodan za brzu i pouzdanu konvergenciju Newton-Raphson rješavača, što je i potvrđeno testiranjem na IEEE 9-bus i 14-bus sistemima. Grafički interfejs (Converter i Options tab) korisniku omogućava jednostavno upravljanje cijelim procesom konverzije i rješavanja, bez potrebe za poznavanjem internog formata `.dmodl` fajla.